

Smart Security Gate

Final Documentation

Field	Detail
Team name	B5
Team members	Muhammad Mujhtaba Qaiser, Muhammad Umer Hayat, Jaleel Ur Rehman, Iko-ojo Victor Odaudu
Project title	Smart Security Gate
Course	Advanced Embedded Systems Lab
Date	07 June 2026

1. Team members

Created by: Muji

This project was built as a group prototype for a smart security gate. The table below shows the final team members and what each person was mainly responsible for during the project.

Team member	Role in project	Main contribution
Muji	Project coordination and documentation	Made the project plan, kept the task board up to date, helped put together the test cases to make sure the prototype worked, and assembled the final report.
Victor	Hardware integration	Wired up the Radio Frequency Identification (RFID) reader, the ultrasonic sensor, the servo, the LEDs, the buzzer and the LCD screen, and sorted out the pin assignments and power.
Jaleel	Embedded software	Wrote the access-control logic, the gate sequence, the sensor reading and the parking counter that runs on the ESP32.
Umer	Communication and testing	Set up the Wi-Fi and Message Queuing Telemetry Transport (MQTT) link and the Raspberry Pi dashboard, and evaluated the prototype behaviour during testing.

2. Introduction

Created by: Umer

The Internet of Things (IoT) is the idea of giving everyday physical objects sensors, actuators, small controllers and a way to communicate, so that they can collect data, share their status and react to what is happening around them. When it comes to access control, this means a gate can be a lot more than just a mechanical barrier. It can detect people, check whether they are allowed in, report what is going on from a distance and keep a digital record of who tried to get in.

A Wireless Sensor Network (WSN) is basically a group of sensor nodes spread out in different places that watch what is happening and send that information back to a controller or a central system. In our project the sensor network is fairly small, which is fine for a prototype. It is made up of a sensor that notices when something is in front of the gate, an RFID reader for the access card, and a wireless link to a dashboard. The point is that the gate is not working on its own, it is part of a connected system that can be monitored.

The target use case for this project is a smart security gate for getting into a controlled area, like a car park or a lab. Someone comes up to the gate, the system notices them, reads their access card, decides if they are allowed in, and then either opens the gate or keeps it locked. On top of that the prototype gives feedback right there at the gate using

LEDs, a buzzer and a small screen, it keeps track of how many parking spaces are still free, and it sends its status wirelessly to a dashboard so it can be watched remotely.

For the actual build we used an ESP32 microcontroller, an RC522 RFID reader for the access card, an ultrasonic sensor to detect a vehicle, a servo motor for the gate arm, LEDs and a buzzer for feedback, a 16x2 Liquid Crystal Display (LCD) for messages at the gate, and a Wi-Fi and MQTT link to a Raspberry Pi that runs the dashboard.

3. Concept description

Created by: Jaleel

The main job of the prototype is to handle access control on its own. It stays locked by default, notices when a model vehicle is at the entrance, asks for a valid card and only opens the gate when the card checks out and a vehicle is actually there. If someone uses a card that is not allowed, the attempt gets rejected and this is shown right at the gate using the red LED, the buzzer and the screen.

The ESP32 is the brain in the middle. It takes in data from the RFID reader and the ultrasonic sensor, runs the logic that decides what to do, controls the servo gate and the feedback parts, keeps count of the free parking spaces, and sends its status over to the Raspberry Pi dashboard through an MQTT broker, which is basically a middleman that passes the messages along.

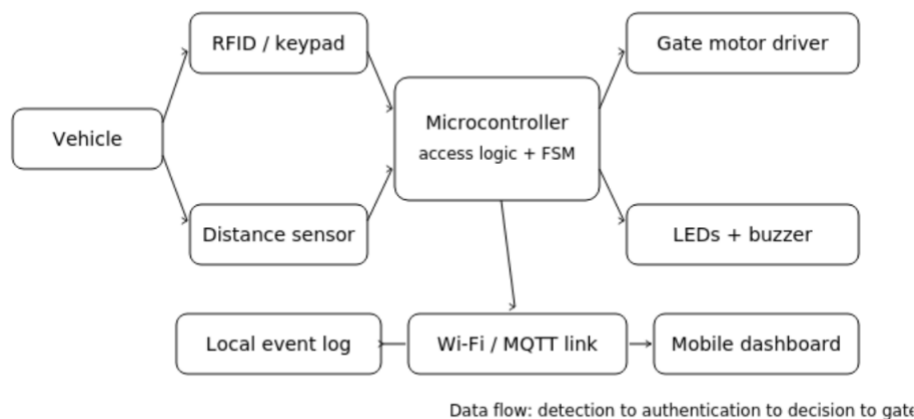


Figure 1: Block diagram of the smart security gate prototype.

The prototype can be split into these main parts and modules:

Component / module	What it is for	Interface used
ESP32 microcontroller	Runs the main program, the access decision logic and the parking counter, and handles the Wi-Fi and MQTT side.	Digital I/O, PWM, SPI, Wi-Fi
RC522 RFID reader	Reads the user's access card, which each have their own unique ID, for the authentication.	SPI (3.3V)
Ultrasonic sensor	Checks that a vehicle is actually in front of the gate before opening, and that it has passed through before closing.	GPIO trigger/echo
Servo motor	Moves the gate arm between closed (0°) and open (90°).	PWM signal
Dual-colour LED	Shows green for access granted and red for denied.	Digital output pins
Buzzer	Gives one beep for granted and two beeps for denied.	Digital output pin
16x2 LCD screen	Shows messages at the gate and the number of free parking spaces.	Parallel (4-bit) GPIO

Component / module	What it is for	Interface used
Push button	Adds a parking space back when a vehicle leaves.	Digital input (pull-up)
Raspberry Pi + dashboard	Runs a web dashboard that shows the gate state and free spaces, and can send a manual open command.	Wi-Fi with MQTT broker

4. Project and team management

Created by: Muji

We managed the project using a light agile approach. Instead of planning the whole thing in one go at the start, we worked in small steps. First we got the gate to move, then sensor reading, then the card authentication, then the wireless side and finally putting it all together and testing. This works well for embedded systems because hardware problems tend to show up when you actually test things, so you need to be able to go back and fix them quickly.

These were the project phases we followed:

1. Requirements and concept: work out how the gate should behave, which cards are allowed, what should happen for safety and how we would know it was working.
2. Hardware bring-up: test each sensor and part on its own with a small sketch before joining them together.
3. Software modules: write the sensor reading, the access checking, the gate control, the feedback and the parking counter.
4. Integration: bring all the modules together into one program on the ESP32.
5. Communication: add the Wi-Fi and MQTT link and the dashboard once the local behaviour was stable.
6. Testing and documentation: run the test cases, take photos and screenshots and write up the report.

Here is the task breakdown and who was responsible for each part:

Work package	Description	Responsible	Status
WP1 Requirements	Work out the use cases, access rules and safety needs.	Muji	Done
WP2 Hardware	Wire up the RC522 RFID reader, ultrasonic sensor, servo, LEDs, buzzer and LCD.	Victor	Done
WP3 Embedded software	Write the gate control logic, the sequence and the parking counter.	Jaleel	Done
WP4 Communication	Send the status and access events to the dashboard using MQTT.	Umer	Done
WP5 Testing	Prepare the test cases, check the response and collect photos and screenshots.	Umer	Done
WP6 Documentation	Write the report with the diagrams, results and references.	Muji	Done

We kept a simple Kanban board with the columns To Do, In Progress, Testing and Done. Each task had one person in charge of it, but anything to do with integration was checked by at least one other person, because a small mistake in the wiring, the pin choices or the communication settings can end up affecting the whole system.

5. Technologies

Created by: Jaleel

The prototype uses fairly standard embedded and IoT parts. The list below is the actual hardware we ended up using in the final build.

5.1 Sensors and inputs

- RC522 RFID reader: this is what reads the access card. It talks to the ESP32 using the Serial Peripheral Interface (SPI) and reads the unique ID (UID) of each card. One important thing is that it runs off the 3.3V pin, not 5V, because 5V can damage it.

- Ultrasonic sensor: this checks whether a vehicle is in front of the gate. It works out distance by sending a pulse and timing how long the echo takes to come back, a bit like how a bat finds things in the dark. We use it twice in the sequence, once to check a vehicle has arrived before opening, and once to check it has gone through before closing.
- Push button: this is an extra input that adds one back to the free parking count when a vehicle leaves the area.

5.2 Actuators and feedback

- Servo motor: this moves the gate arm. It is a good fit for a lab prototype because it can hold set angles, so 0° for closed and 90° for open.
- Dual-colour LED: this shows the status, green for access granted and red for denied.
- Buzzer: this gives quick sound feedback, one beep for granted and two beeps for denied, so you get a response even without looking at the screen.
- 16x2 LCD screen: this shows short messages like Scan card, Access Granted and Access Denied, along with the current number of free parking spaces.

5.3 Communication protocols and programming

- The ESP32 program is written in C/C++ using the Arduino IDE.
- SPI is used for the RC522 RFID reader, Pulse Width Modulation (PWM) is used for the servo, and normal General Purpose Input/Output (GPIO) pins are used for the LEDs, buzzer, LCD, ultrasonic sensor and button.
- The status data is sent over Wi-Fi using MQTT, which is a lightweight publish and subscribe protocol made for small devices and networks that do not have a lot of bandwidth. The ESP32 publishes the parking count and the Raspberry Pi subscribes to it and updates the dashboard.
- An MQTT broker called Mosquitto runs on the Raspberry Pi and acts as the message hub between the ESP32 and the dashboard, so the two do not have to talk to each other directly.
- Serial output was used a lot during development to check sensor values, card IDs and the access decisions while debugging.

5.4 Design decision: ESP32 instead of Arduino Uno WiFi

At the start we tried to use an Arduino Uno WiFi as the main controller. The problem was that it would not communicate reliably with the RC522 RFID reader over SPI, and the reader kept returning an invalid response no matter what we tried. After a fair bit of testing we switched to the ESP32, which talked to the reader without any issues and also had Wi-Fi built in for the MQTT link. That made the ESP32 both the simpler and the more reliable option, so it became the one microcontroller running the whole system.

5.5 Security considerations

- We do not store any real personal data in the prototype, only test card IDs.
- The allowed IDs are written straight into the code for this prototype. That is fine for a demo as long as it is clearly stated, but a real system would use a proper secure database instead.
- The gate fails safe, so if the power is lost or something goes wrong, the servo stays in the closed position and the software will not open the gate by accident.

6. Implementation

Created by: Victor

The implementation is built around one main loop running on the ESP32. The system moves through a clear set of steps. It stays idle and locked, waits for a valid card, then waits for a vehicle to actually pull up before opening, and

finally closes the gate once the vehicle has gone through. Doing it this way keeps things predictable because the gate is only ever doing one thing at a time. The diagram below shows how it moves between these states.

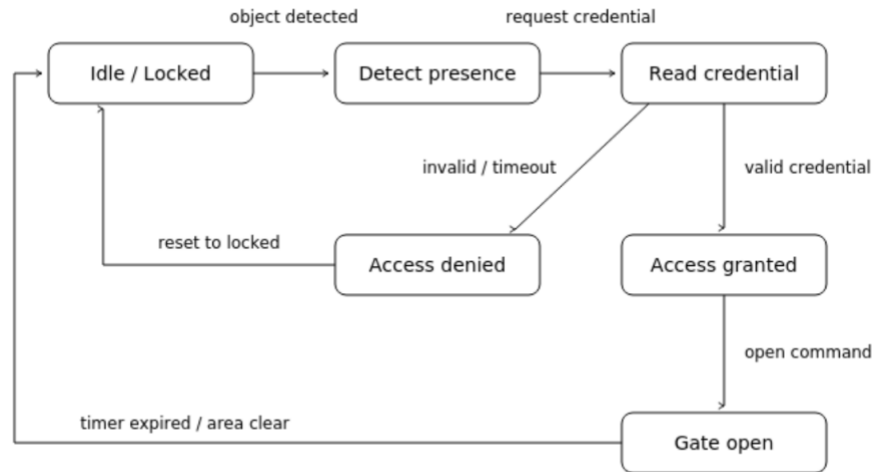


Figure 2: State diagram of the smart security gate behaviour.

6.1 Hardware wiring and pin connections

Everything is run by a single ESP32 DevKitC. We spread the parts across two breadboards so that related components stay together and it does not get too crowded. Board A has the RFID reader and the LCD, and Board B has the LEDs, buzzer and ultrasonic sensor. Since there is only one ESP32, the power and ground rails of the two breadboards are joined together, red to red and blue to blue, so both boards get power from the same supply.

There are two power voltages in use. Most parts run off the 5V rail, but the RC522 RFID reader is powered straight from the ESP32's 3.3V pin. This one really matters, because giving the RC522 5V can damage it, so it is the one part that must not go on the shared 5V rail. Everything shares a common ground.

The table below lists every connection between a component and the ESP32:

Component	Component pin	ESP32 pin	Notes
RC522 RFID reader	SDA (SS)	GPIO 5	SPI chip select
RC522 RFID reader	SCK	GPIO 18	SPI clock
RC522 RFID reader	MOSI	GPIO 23	SPI data out
RC522 RFID reader	MISO	GPIO 19	SPI data in
RC522 RFID reader	RST	GPIO 27	Reset
RC522 RFID reader	3.3V	3.3V pin	Direct, not the 5V rail
RC522 RFID reader	GND	GND rail	Common ground
Servo (gate)	Signal	GPIO 14	PWM control
Servo (gate)	Power / GND	5V rail / GND	Red and brown wires
Green LED	Signal	GPIO 15	Access granted
Red LED	Signal	GPIO 25	Access denied
Buzzer	Signal	GPIO 4	One or two beeps
Ultrasonic sensor	Trig	GPIO 32	Sends the pulse
Ultrasonic sensor	Echo	GPIO 33	Receives the echo
LCD (16x2)	RS	GPIO 21	Register select
LCD (16x2)	E	GPIO 22	Enable
LCD (16x2)	D4	GPIO 16	Data line
LCD (16x2)	D5	GPIO 17	Data line
LCD (16x2)	D6	GPIO 26	Data line
LCD (16x2)	D7	GPIO 13	Data line

Component	Component pin	ESP32 pin	Notes
Push button	One leg	GPIO 12	Other leg to GND

The LCD is wired up in 4-bit mode, which uses four data lines (D4 to D7) instead of eight, so it saves us some pins. While putting it together we had to pick some of the pins carefully. A few of the ESP32 pins, like GPIO 12, are strapping pins that the board checks when it first powers on, and if the LCD was connected to them it stopped new code from uploading. So we put the button on GPIO 12, which is only read after start-up, and moved the LCD data lines onto pins that do not get in the way of the boot process. Figuring this out was one of the more useful practical lessons during the build.

Here is a summary of how the ESP32 pins are used overall:

ESP32 pin	Used for
GPIO 5, 18, 23, 19, 27	RC522 RFID reader (SPI and reset)
GPIO 14	Servo (gate) signal
GPIO 15 / 25	Green LED / Red LED
GPIO 4	Buzzer
GPIO 32 / 33	Ultrasonic sensor (Trig / Echo)
GPIO 21, 22, 16, 17, 26, 13	LCD (RS, E, D4 to D7)
GPIO 12	Leaving button
3.3V pin	RC522 power (direct)
5V rail	Servo, LEDs, buzzer, ultrasonic, LCD power

Altogether the project uses about 15 of the ESP32's roughly 25 usable GPIO pins, so there is still room left for future additions like a second gate or more sensors.

6.2 The Raspberry Pi and dashboard

The Raspberry Pi has two jobs in this project. It runs the web dashboard that lets you monitor the system from a browser, and it also runs the MQTT broker, which is the message hub the ESP32 and the dashboard use to talk to each other.

The dashboard itself is a small web page served by a Python tool called Flask, which runs on the Pi. Anyone on the same network can open it in a browser and see the current status of the system. It shows how many parking spaces are free, with a small bar that fills up or empties, and it has a manual button that can open the gate straight from the browser. The page refreshes itself every few seconds so the numbers stay up to date, a bit like an arrivals board at an airport that keeps updating on its own.

Getting the Pi and the ESP32 to talk wirelessly was one of the trickier parts. We were using a phone hotspot for the network, and hotspots usually block the devices on them from contacting each other directly, which is a privacy feature. Our first attempt, where the Pi tried to reach the ESP32 directly, did not work for this reason. The MQTT broker on the Pi solved it, because instead of the two devices reaching each other directly, they both just talk to the broker. The ESP32 posts its parking count to the broker, and the Pi reads it from there and updates the dashboard. It is like two people who cannot call each other directly but can both use the same group chat.

So the flow is: the ESP32 does all the physical work at the gate and posts updates to the broker, and the Pi listens to the broker and shows everything on the dashboard. Because they are only ever talking to the broker and not to each other directly, the whole thing works wirelessly even on the hotspot.

6.3 Static structure of the software

The ESP32 program is split into functions, and each one has a clear job:

Function	What it does
setup()	Runs once when the ESP32 powers on. It gets the RFID reader, servo, LEDs, buzzer, LCD and button ready, and connects to Wi-Fi and the MQTT broker.
loop()	Runs over and over. It keeps the MQTT connection alive, checks the leaving button, waits for a card and then runs the entry sequence.
checkAccess()	Compares the scanned card ID against the list of allowed cards and gives back a yes or no.

Function	What it does
getDistance()	Reads the ultrasonic sensor and gives back how far away the nearest object is, in centimetres.
openGate()	The full let-them-in routine: green LED on, one beep, the servo opens, the parking count drops by one, it waits for the vehicle to pass and then the servo closes.
denyAccess()	The turn-them-away routine: red LED on, two beeps, Access Denied on the screen and the gate stays shut.
showReady()	Puts the LCD back to the idle message and shows the current number of free spaces.
publishSpots()	Sends the current parking count to the MQTT broker so the dashboard stays up to date.
onMessage()	Deals with a manual OPEN command that comes in from the dashboard through MQTT.
reconnectMQTT()	Keeps the connection to the broker going and reconnects on its own if it drops.

6.4 Use cases

Use case	Actor	Main flow	Result
Authorised entry	User with a valid card	The card is scanned and recognised, a vehicle is detected at the gate, the servo opens, the parking count drops by one and the dashboard updates.	Gate opens and then closes once the vehicle has passed.
Unauthorised attempt	User with an invalid card	The card is scanned but it is not on the allowed list.	Gate stays closed, red LED and two beeps, and Access Denied on the screen.
Valid card, no vehicle	Valid card but no vehicle there	The card is recognised but the ultrasonic sensor does not see a vehicle within the time limit.	Gate stays closed and the system goes back to idle.
Parking full	Valid card when the count is 0	The card is recognised but there are no spaces left.	Gate stays closed and the screen shows Parking FULL.
Vehicle leaves	Attendant	The leaving button gets pressed.	Parking count goes up by one, up to the maximum of 7, and the dashboard updates.
Remote monitoring	Operator or team member	The dashboard reads the state that was published over MQTT.	The operator can see the live parking count and can open the gate manually.

6.5 Pseudo-code for the main loop

```

start with parkingSpots = 7, gate CLOSED
loop:
  keep MQTT connection alive
  if leaving button pressed and parkingSpots < 7:
    parkingSpots = parkingSpots + 1; update dashboard
  if a card is scanned:
    if card is authorised:
      wait up to 10s for a vehicle at the gate
      if vehicle detected and parkingSpots > 0:
        green LED on; one beep; open gate
        parkingSpots = parkingSpots - 1; update dashboard
        wait until vehicle has passed; close gate

```

```

else if parkingSpots == 0:
    show "Parking FULL"
else:
    show timeout; return to idle
else:
    red LED on; two beeps; show "Access Denied"

```

7. Results

Created by: Umer

We put the full system together and tested it, and it all worked as we wanted. Every part worked together, and the wireless link between the ESP32 and the Raspberry Pi dashboard was up and running.

7.1 Evaluation criteria and outcomes

Test case	Expected result	Outcome
Valid credential	Gate opens, green LED on, one beep, logged as granted.	Passed. The gate opened and the green LED came on for allowed cards.
Invalid credential	Gate stays closed, red LED and buzzer, logged as denied.	Passed. The gate stayed closed with the red LED and two beeps.
Vehicle detection	Gate only opens when a vehicle is actually at the gate.	Passed. The gate waited for the ultrasonic sensor to see a vehicle before opening.
Parking counter	Count goes down on entry and up when the button is pressed.	Passed. The count dropped on entry and was put back by the button, up to a max of 7.
Parking full	Gate does not open when the count reaches zero.	Passed. Parking FULL was shown and the gate stayed shut.
Dashboard update	Dashboard shows the live parking count over MQTT.	Passed. The dashboard updated on its own after each entry and exit.

7.2 Result narrative

During the integration testing the ESP32 read the access card properly, and for an allowed card the servo moved the gate to the open position while the green LED and a single beep showed it had worked. For a card that was not allowed, the gate stayed closed and the buzzer gave two short beeps along with the red LED and an Access Denied message on the screen. The ultrasonic sensor correctly made sure a vehicle was there before the gate opened, and checked it had passed through before closing. The parking counter went down on each entry and was put back using the leaving button, and the current count showed up both on the LCD and on the dashboard. The dashboard on the Raspberry Pi got the parking count over the MQTT broker and updated by itself.

Note: photos of the finished prototype, a close-up of the wiring, a screenshot of the serial output and a screenshot of the dashboard should be added here as final evidence before submission.

7.3 Known limitations and possible improvements

- This is a prototype, not a real security product, since simple RFID tags and IDs written into the code can be copied or bypassed.
- We were on a phone hotspot, which blocks devices from talking to each other directly. We got around this by sending everything through the MQTT broker on the Pi, but a normal router network would be more reliable.
- How reliable the gate is depends a lot on the gate arm, the servo strength and having a steady power supply.
- The ultrasonic sensor threshold has to be set for the test area so it does not trigger by mistake.
- The LED does not have a current-limiting resistor in the current build, which is okay for a short demo but not great for long-term use.

- Future improvements could include encrypted communication, a proper card database, a mobile app, camera checking, a battery backup and physical limit switches on the gate.

8. Sources / References

Created by: Victor

The project repository link and the exact component datasheets should be added here before final submission. The lists below cover the standards, libraries, drivers, tools and tutorials that were actually used during the build.

8.1 Standards and protocols

1. OASIS Open, "MQTT Version 5.0," official standard documentation. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
2. MQTT.org, "MQTT Specifications." <https://mqtt.org/mqtt-specification/>

8.2 Hardware documentation

3. Espressif Systems, "ESP32 Wi-Fi & Bluetooth SoC Documentation." <https://www.espressif.com/en/products/socs/esp32/documentation>
4. Arduino Documentation, "Servo." <https://www.arduino.cc/en/reference/servo>
5. Arduino Docs, "Servo Motor Basics with Arduino." <https://docs.arduino.cc/learn/electronics/servo-motors/>
6. ArduinoModules.info, "KY-029 Dual Color LED Module." <https://arduinomodules.info/ky-029-dual-color-led-module/>

8.3 Arduino libraries (installed via the Library Manager)

7. MFRC522 RFID library. GitHub, miguelbalboa/rfid, "Arduino RFID Library for MFRC522." <https://github.com/miguelbalboa/rfid>
8. ESP32Servo library, used to control the servo on the ESP32. <https://github.com/madhephaestus/ESP32Servo>
9. LiquidCrystal library, used to drive the 16x2 LCD in 4-bit mode. <https://docs.arduino.cc/libraries/liquidcrystal/>
10. PubSubClient library by Nick O'Leary, used for the MQTT connection on the ESP32. <https://github.com/knolleary/pubsubclient>

8.4 ESP32 board support and drivers

11. Arduino core for the ESP32 (Espressif), added through the Boards Manager using the Espressif board package URL. <https://github.com/espressif/arduino-esp32>
12. Silicon Labs CP210x USB-to-UART driver, needed on the Mac so it could recognise the ESP32 over USB. <https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers>

8.5 Raspberry Pi and dashboard software

13. Flask, the Python web framework used to run the dashboard on the Raspberry Pi. <https://flask.palletsprojects.com/>
14. pyserial, the Python library for serial communication that was set up on the Pi. <https://pyserial.readthedocs.io/>
15. paho-mqtt, the Python MQTT client used by the dashboard to talk to the broker. <https://pypi.org/project/paho-mqtt/>
16. Eclipse Mosquitto, the open source MQTT broker that runs on the Raspberry Pi. <https://mosquitto.org/>

8.6 Development tools

17. Arduino IDE, used to write and upload the code to the ESP32. <https://www.arduino.cc/en/software>